Lesson 10 — A Front End and Deployment

Node.js/Express

Game Plan

- Warm-Up
- The React Front End
- What Goes Into Internet Deployment
- Cloud Database with Neon
- Deploying to Render.com
- Pre-Deployment Security
- Assignment Preview
- Wrap-Up

Warm-Up (5 min)

In chat or out loud:

1. You've been using `http://localhost:3000` this whole course. What would need to change to make your app available to anyone on the Internet?
2. Have you ever deployed an app before? What was the hardest part?

You Built a Working Back End

Over the last 9 weeks, your app has:

- User registration, login, logout with JWT auth
- Full CRUD for tasks (protected by auth)
- PostgreSQL database with Prisma
- Input validation and password hashing
- Security middleware (helmet, rate limiting, XSS protection)
- Automated tests

Now it's time to put it on the Internet.

The React Front End

A React front end has been provided for you.

It does everything the course project needs:

- User register, login, logout
- List, create, update, delete tasks
- Sends credentials with `credentials: "include"` (forwards the cookie)
- Sends the CSRF token in headers for write requests
- Uses local storage for the CSRF token and username — not the JWT

You don't need to modify the front end.

Why You Can't Use localhost for Production

Your local database is at `localhost:5432` — no one else can reach it.

Your local server is at `localhost:3000` — ditto.

For the Internet:

- Database needs to be **cloud-hosted** (Neon)
- App needs to be **cloud-hosted** (Render.com)
- Both need **public URLs**

What Internet Deployment Needs

1. **Server hardware** (or a cloud service)
2. **Public IP address**
3. **DNS name** (yourapp.onrender.com)
4. **SSL certificate** (HTTPS — provided by Render automatically)
5. **Cloud database** (Neon — cloud Postgres)
6. **Environment variables** (secrets don't go in Git)
7. **Automated deployment** (push to GitHub → Render redeploys)

Cloud providers handle most of this for free at small scale.

Step 1: Cloud Database with Neon

Neon (neon.tech) provides free hosted PostgreSQL.

After creating your database:

1. Copy the connection string — it's your new `DATABASE_URL`
2. Update `.env` (locally, for testing)
3. Run `npx prisma migrate deploy` to create the tables
4. Test with Postman against the Neon database

Your local data is **not** automatically in Neon — you'll need to re-create test users.

Step 2: Deploy to Render.com

Render.com gives you free Node.js hosting connected to GitHub.

Configure:

- **Build command:** `npm install && npx prisma migrate deploy`
- **Start command:** `npm start`
- **Environment variables:** Set `DATABASE_URL` and `JWT_SECRET` in Render's dashboard

Push to GitHub → Render redeploys automatically.

The Deployment Flow

```
Local code  →  GitHub (push)  →  Render.com (auto-build)
                                          ↓
                                    npm install
                                    prisma migrate deploy
                                    npm start
                                          ↓
                                App is live at yourapp.onrender.com
```

Environment Variables in Production

`.env` is in `.gitignore` — it never goes to GitHub.

So how does Render know your secrets?

→ Set them in Render's dashboard under **Environment Variables**.

Same pattern for any cloud service: secrets live in the platform's config, never in your code repository.

One Remaining Security Gap

The `/api/users/register` route has no protection:

- Anyone can register unlimited fake accounts
- No verification that the email is real

Solution for this week: Add Google reCaptcha to prevent bot registrations.

This protects against automated attacks without requiring email verification.

Pre-Deployment Security Checklist

Before going live, verify:

- ✅ JWT secret is in `.env`, not hardcoded
- ✅ Database URL is in `.env`, not hardcoded
- ✅ Helmet middleware is active
- ✅ Rate limiting is on
- ✅ XSS sanitizer is on
- ✅ Passwords are hashed (not stored plain)
- ✅ CSRF protection on all write routes

We Do — Trace a Production Request

A user visits `yourapp.onrender.com` and logs in. Trace the path:

1. Browser → React front end (served from where?)
2. Login form → `POST /api/users/logon` (which server?)
3. Server validates credentials (which database?)
4. Server sets JWT cookie
5. Subsequent task request sends cookie along

You Do (5 min)

Look at these deployment gotchas. Which ones have you seen or might you encounter?

1. App crashes on Render — how do you see the error logs?
2. Postman works locally but not on Render — what might be different?
3. Database connection fails — what should you check first?

What the Free Tier Means

On Render's free plan:

- **Builds are slow** — be patient
- **App sleeps after 15 minutes of inactivity**
- **Wakes up slowly** — first request after sleep may take 30+ seconds

Don't demo from Render's cold start! Hit the URL a few minutes before your presentation.

Assignment Preview

Assignment 10:

1. Set up a Neon cloud database
2. Run Prisma migrations against Neon
3. Deploy your app to Render.com
4. Configure the React front end to point to your Render URL
5. Add reCaptcha to the register route
6. Test everything end-to-end (Postman + the React front end)

Wrap-Up

In chat:

1. Why can't you use your local database in a cloud deployment?
2. Where do production secrets (like `JWT_SECRET`) go if not in your code?
3. What does Render's build command do before starting the app?

Confidence Check

On a scale of 1–5:

How comfortable do you feel deploying your app this week?

Resources

- https://neon.tech/docs
- https://render.com/docs/deploy-node-express-app
- https://www.prisma.io/docs/guides/deployment
- Ask questions in Slack

Closing

This week:

Your app goes live on the Internet.

Next week:

The capstone — add something extra, learn how to start your own project, and wrap up the course.

You've built a real web API. That's no small thing. See you at the finish line!